

Assignment_1

Mingjie Wei

Setup

```
# You will need the correct file paths if you don't follow my naming conventions:
file_a <- "../texts/A07594__Circle_of_Commerce.txt"
file_b <- "../texts/B14801__Free_Trade.txt"

# Read the raw text files into R
text_a <- read_file(file_a)
text_b <- read_file(file_b)

# Combine into a tibble for tidytext workflows
texts <- tibble(
  doc_title = c("Text A", "Text B"),
  text = c(text_a, text_b)
)

texts
```

```
# A tibble: 2 x 2
  doc_title text
  <chr>      <chr>
1 Text A    THE CIRCLE OF COMMERCE. The Prooeme. HERODOTVS in his CLI0, reporte~
2 Text B    CAP. I. The Causes of the want of Money in England. IT hauing pleas~
```

Diagnostics (before stopword removal)

Create a tibble called `corpus_diagnostics` with one row per document that includes:

- `doc_title`
- `n_chars` = number of characters in each document

- `n_word_tokens` = number of word tokens before stopwords removal
- `n_word_types` = number of unique word types after lowercasing, before stopwords removal

```
chars_tbl <- texts %>%
  mutate(n_chars = nchar(text)) %>%
  select(doc_title, n_chars)

tokens_tbl <- texts %>%
  unnest_tokens(word, text) %>%
  mutate(word = str_to_lower(word))

token_diag_tbl <- tokens_tbl %>%
  group_by(doc_title) %>%
  summarise(
    n_word_tokens = n(),
    n_word_types = n_distinct(word),
    .groups = "drop"
  )

corpus_diagnostics <- chars_tbl %>%
  left_join(token_diag_tbl, by = "doc_title")

corpus_diagnostics
```

```
# A tibble: 2 x 4
  doc_title n_chars n_word_tokens n_word_types
  <chr>      <int>      <int>      <int>
1 Text A    191605      33889      4773
2 Text B    114211      19922      3332
```

Tokenization, stopwords, and counting words

1. Stopwords

```
# Start with tidytext's built-in stopwords list
data("stop_words")

# Add our own project-specific stopwords (you can, and will, expand this list later)
custom_stopwords <- tibble(
  word = c(
```

```

    "vnto", "haue", "doo", "hath", "bee", "ye", "thee"
  )
)

all_stopwords <- bind_rows(stop_words, custom_stopwords) %>%
  distinct(word)

all_stopwords %>% slice(1:10)

```

```

# A tibble: 10 x 1
  word
  <chr>
1 a
2 a's
3 able
4 about
5 above
6 according
7 accordingly
8 across
9 actually
10 after

```

2. Tokenization

```

word_counts <- texts %>%
  unnest_tokens(word, text) %>%
  mutate(word = str_to_lower(word)) %>%
  anti_join(all_stopwords, by = "word") %>%
  count(doc_title, word, sort = TRUE)

word_counts

```

```

# A tibble: 7,443 x 3
  doc_title word      n
  <chr>      <chr>  <int>
1 Text A    trade    232
2 Text A    exchange  186
3 Text B    trade    185

```

```

4 Text A    fo          184
5 Text A    malynes     158
6 Text A    merchants   126
7 Text A    mony        118
8 Text A    hee         115
9 Text A    kingdome    113
10 Text A   commodities  96
# i 7,433 more rows

```

Normalize word frequencies (after stopwords removal)

```

doc_lengths <- word_counts %>%
  group_by(doc_title) %>%
  summarise(total_words = sum(n), .groups = "drop")

word_counts_normalized <- word_counts %>%
  left_join(doc_lengths, by = "doc_title") %>%
  mutate(relative_freq = n / total_words)

# Compare "trade"
word_counts_normalized %>%
  filter(word == "trade") %>%
  select(doc_title, n, total_words, relative_freq)

```

```

# A tibble: 2 x 4
  doc_title      n total_words relative_freq
  <chr>      <int>      <int>      <dbl>
1 Text A      232      14694      0.0158
2 Text B      185       8004      0.0231

```

Comparing word frequencies across texts

```

plot_n_words <- 20

word_comparison_tbl <- word_counts_normalized %>%
  select(doc_title, word, relative_freq) %>%
  pivot_wider(
    names_from = doc_title,

```

```

      values_from = relative_freq,
      values_fill = 0
    ) %>%
    mutate(max_rf = pmax(`Text A`, `Text B`)) %>%
    arrange(desc(max_rf))

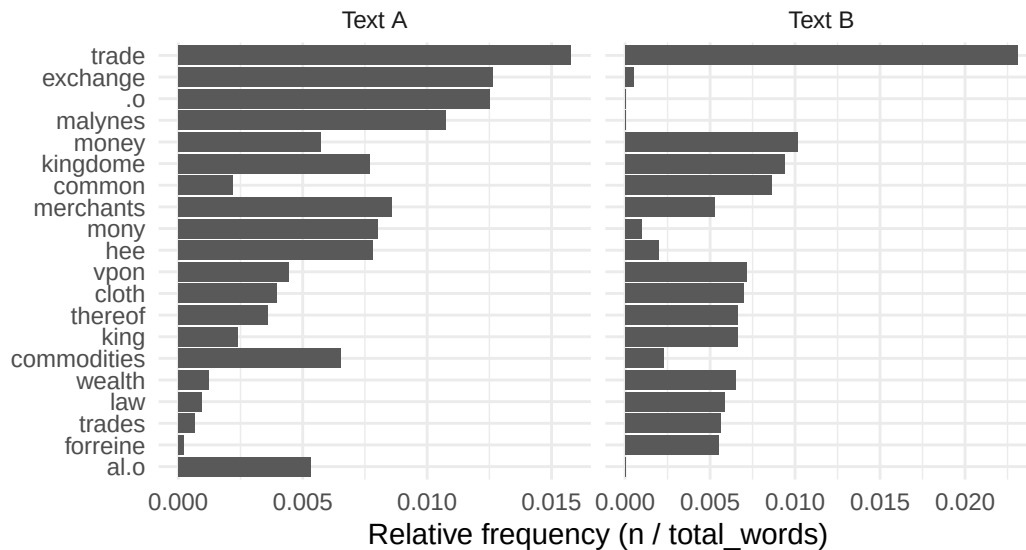
word_plot_data <- word_comparison_tbl %>%
  slice_head(n = plot_n_words) %>%
  pivot_longer(
    cols = c(`Text A`, `Text B`),
    names_to = "doc_title",
    values_to = "relative_freq"
  ) %>%
  mutate(word = fct_reorder(word, relative_freq, .fun = max))

ggplot(word_plot_data, aes(x = relative_freq, y = word)) +
  geom_col() +
  facet_wrap(~ doc_title, scales = "free_x") +
  labs(
    title = "Most frequent words (normalized; stopwords removed)",
    subtitle = paste0(
      "Top ", plot_n_words,
      " words by maximum relative frequency across both texts"
    ),
    x = "Relative frequency (n / total_words)",
    y = NULL
  ) +
  theme_minimal()

```

Most frequent words (normalized; stopwords removed)

Top 20 words by maximum relative frequency across both texts



Bigrams: starting to think about context

- bigrams

```
bigrams <- texts %>%
  unnest_tokens(bigram, text, token = "ngrams", n = 2)

bigrams
```

```
# A tibble: 53,809 x 2
  doc_title bigram
  <chr>      <chr>
1 Text A    the circle
2 Text A    circle of
3 Text A    of commerce
4 Text A    commerce the
5 Text A    the prooeme
6 Text A    prooeme herodotvs
7 Text A    herodotvs in
8 Text A    in his
9 Text A    his clio
10 Text A   clio reportes
# i 53,799 more rows
```

- `separate()` takes the bigram column and splits each string at the space character.

```
bigrams_separated <- bigrams %>%
  separate(bigram, into = c("word1", "word2"), sep = " ")

bigrams_separated
```

```
# A tibble: 53,809 x 3
  doc_title word1      word2
  <chr>      <chr>      <chr>
1 Text A    the        circle
2 Text A    circle      of
3 Text A    of          commerce
4 Text A    commerce    the
5 Text A    the         prooeme
6 Text A    prooeme     herodotvs
7 Text A    herodotvs   in
8 Text A    in          his
9 Text A    his         clio
10 Text A   clio        reportes
# i 53,799 more rows
```

- remove `all_stopwords` (we remove bigrams where *either* word is a stopword, since phrases like “of the” or “and the” are rarely meaningful analytically.)

```
bigrams_filtered <- bigrams_separated %>%
  filter(
    !word1 %in% all_stopwords$word,
    !word2 %in% all_stopwords$word
  )

bigrams_filtered
```

```
# A tibble: 7,622 x 3
  doc_title word1      word2
  <chr>      <chr>      <chr>
1 Text A    prooeme     herodotvs
2 Text A    clio        reportes
3 Text A    croesvs     king
4 Text A    fonne       borne
5 Text A    borne       dumbe
```

```

6 Text A    king    himself
7 Text A    imminent danger
8 Text A    certaine perfian
9 Text A    perfian ready
10 Text A    lay      violent
# i 7,612 more rows

```

- group by + count

```

bigram_counts <- bigrams_filtered %>%
  count(doc_title, word1, word2, sort = TRUE)

```

```
bigram_counts
```

```

# A tibble: 6,449 x 4
  doc_title word1    word2      n
  <chr>      <chr>    <chr>   <int>
1 Text B    common  wealth    48
2 Text A    merchants adventurers 35
3 Text B    latin   alphabet  34
4 Text A    low     countries 21
5 Text B    east    india     20
6 Text A    free    trade     19
7 Text A    latin   alphabet  17
8 Text A    native commodities 17
9 Text A    forraine commodities 16
10 Text A    letters patents   14
# i 6,439 more rows

```

- Now that we removed the stopwords, we can use `unite` to put the bigrams back together

```

bigram_counts <- bigram_counts %>%
  unite(bigram, word1, word2, sep = " ")

```

```
bigram_counts
```

```

# A tibble: 6,449 x 3
  doc_title bigram      n
  <chr>      <chr>   <int>
1 Text B    common wealth    48
2 Text A    merchants adventurers 35
3 Text B    latin alphabet  34

```



```

4 Text A    low countries      21
5 Text B    east india        20
6 Text A    free trade        19
7 Text A    latin alphabet    17
8 Text A    native commodities 17
9 Text A    forraine commodities 16
10 Text A   letters patents    14
# i 6,439 more rows

```

Comparing bigrams

- normalize by the number of bigrams *within* each document to compare the two + transposition

```

bigram_relative <- bigram_counts %>%
  group_by(doc_title) %>%
  mutate(
    total_bigrams = sum(n),
    proportion = n / total_bigrams
  ) %>%
  ungroup()

bigram_wide <- bigram_relative %>%
  select(doc_title, bigram, proportion) %>%
  pivot_wider(
    names_from = doc_title,
    values_from = proportion,
    values_fill = 0
  )

bigram_wide

```

```

# A tibble: 6,379 x 3
  bigram          `Text B` `Text A`
  <chr>          <dbl>   <dbl>
1 common wealth    0.0187    0.00257
2 merchants adventurers 0      0.00692
3 latin alphabet   0.0133    0.00336
4 low countries    0.00156    0.00415
5 east india       0.00780     0
6 free trade       0      0.00376

```

```

7 native commodities      0.00273  0.00336
8 forraine commodities    0          0.00316
9 letters patents         0.000390  0.00277
10 thoufand pounds        0          0.00277
# i 6,369 more rows

```

- We now want to know which bigram is most characteristic of “Text A” (*The Circle of Commerce*) relative to “Text B” (*Free Trade*).

```

bigram_diff <- bigram_wide %>%
  mutate(
    diff = `Text A` - `Text B`
  ) %>%
  arrange(desc(abs(diff)))

bigram_diff %>% slice(1:20)

```

```

# A tibble: 20 x 4
  bigram          `Text B` `Text A`      diff
  <chr>          <dbl>   <dbl>   <dbl>
1 common wealth  0.0187  0.00257 -0.0161
2 latin alphabet 0.0133  0.00336 -0.00989
3 east india     0.00780  0        -0.00780
4 merchants adventurers 0        0.00692  0.00692
5 merchants aduenturers 0.00390  0        -0.00390
6 free trade     0        0.00376  0.00376
7 forraine commodities 0        0.00316  0.00316
8 thoufand pounds 0        0.00277  0.00277
9 low countries  0.00156  0.00415  0.00259
10 letters patents 0.000390  0.00277  0.00238
11 publique vtility 0.00234  0        -0.00234
12 cloth trade   0.00429  0.00198 -0.00231
13 33 fh         0        0.00218  0.00218
14 20 fhillings  0        0.00198  0.00198
15 fh 4          0        0.00198  0.00198
16 disorderly trade 0.00195  0        -0.00195
17 fishing vpon  0.00195  0        -0.00195
18 india stocke  0.00195  0        -0.00195
19 kings honour  0.00195  0        -0.00195
20 maiesties subiects 0.00195  0        -0.00195

```

Canvas response

II

Text A has 191,605 characters and 33,889 word tokens, while Text B has 114,211 characters and 19,922 word tokens. Text A is therefore about $1.7\times$ longer than Text B by both characters and tokens, so the two texts are not directly comparable using raw word counts. Text A also has more unique word types (4,773 vs 3,332), which is consistent with it being longer and covering more total vocabulary. Because of the length difference, raw counts (e.g., “trade = 232 vs 185”) can be misleading: higher counts may simply reflect more text rather than greater emphasis. Normalizing by document length (e.g., using relative frequency) makes comparisons fairer by focusing on proportions instead of totals.

III

Q1: Who proportionally uses “trade” more? Does it match raw counts?

In raw counts, Text A uses “trade” more often (232 vs 185). However, after normalizing by document length ($n / \text{total_words}$ after stopwords removal), Text B uses “trade” more proportionally (0.0231 vs 0.0158), about $1.46\times$ higher. This shows that raw counts can be driven by overall document length rather than emphasis.

Q2: What if you normalize by length before stopwords removal?

If we normalize using total word counts before stopwords removal, the denominator includes stopwords (function words). Because texts can differ in stopwords density, the relative frequencies of content words like “trade” could be diluted by different amounts across texts, potentially shifting comparisons. Normalizing after stopwords removal is more consistent with the object being compared (content-word frequencies), while normalizing before stopwords removal better reflects overall language use including stylistic function words.